Xin Chao Weather Impact; Web App Structure and Future Steps

NAHSS Weather Impact Project Group

March 24, 2026

Abstract

This report serves as both a guide for the web app's functionalities and as an explanation of the extendability of the structure. In the first part of the report, we discuss the functionality of the front-end of the web app. We discuss the different information points the app provides, and the functions the options the user can choose. In the second part of the report, we discuss the back-end structure. We delve into the modelling choices of the back-end and different novel functions we use for obtaining information. We also explain where the functions can be expanded if necessary.

For live interaction with the application, visit the HOI-WI website. For the public GitHub repository of the application, visit Github.

Contents

1 Introduction

Weather Impact (WI) is a Dutch company that provides weather information and crop cultivation advice through the social media application *Telegram* in the form of a chatbot to smallholder farmers in Africa. The information provided aims to help farmers improve their decision-making and increase their knowledge of agriculture.

Currently, Weather Impact is looking for opportunities to expand its services to the country Viet Nam in Asia. The current service provided by WI is a chatbot integrated into the app *Telegram*. This chatbot allows users to choose from a range of options, such as weather forecasts or crop advice and provides the user with information about the selected option. The chatbot structure is decision-tree based; this structure is simple to understand and easily extendable. The current version of the chatbot provides weather information, crop advice, notifications, and information for good agricultural practices. In this report, we present a proposal for the structure of the chatbot HOI-WI[1], for a version with additional features and information more tailored to the needs of the smallholder farmers in Viet Nam.

The *HOI-WI* web application addresses the demand of farmers by offering a chatbot structure that provides weather insights, cultivation advice for crops, price information, and information about alternate farming techniques, tailored to the local context and the farmer's literacy.

In this document, we present the structure and implementation of the back-end architecture that enables these functions. We use multiple public data sources, including the NASA POWER API, OpenWeatherMap, and various price information platforms, to offer accurate and relevant information. Methods Geographic filtering, distance-based clustering heuristics, and dynamic content generation ensure that users receive accurate and location-specific information. Furthermore, we use GitHub as a lightweight cloud storage for processed data and visual outputs, supporting scalability and version control.

The back-end codebase is structured to prioritise modularity, maintainability, comprehensibility, and extensibility, thereby enabling future adaptation to new regions, crops, or external APIs.

We first discuss the chatbot user experience, we delve into the functions and information the user can access. Thereafter, we cover the different underlying code files for this web application. We explain the data gathering process, the file with all functions defined, and the file that connects the code to the data storage for uploading the different information files.

Definition of Frequently Used Terms
In this report, we define frequently used terms as follows: **Front-end**: The front-end of a code refers to the information the user sees. This is aimed at being easily understandable, facile to navigate in, and aesthetically pleasing. **Back-end**: The back-end of the code is the part that takes care of the information handling and the structure of processes. The back-end is not visible to the user, but it ensures that all connections are correct and efficient. **Decision Tree**: A decision tree is a tool used for making decisions based on a series of questions and responses. Each "branch" of the tree represents a choice or test, and each "leaf" at the end shows a result or outcome. It's called a "tree" because its structure resembles an upside-down tree, starting with one main question and branching out into many possible outcomes.

2 Proposed Chatbot: Front-End

The *HOI-WI* web app is a Streamlit-based application designed to show the structure for its chatbot counterpart. It provides smallholder farmers with localised weather data, crop advice, market prices, and information about alternative farming methods. The application loads dynamic content from a public GitHub repository and operates through a option menu-based user interface. In this section, we discuss the information the chatbot provides. Note that the data is not of competitive quality as the primary aim of this project was to provide a new structure and to look for novel ways to gather data, accommodate user needs, and embody the research we have performed.

[1] HOI translates to "ask" in Vietnamese, this is inspired after the name "ULIZA-WI". As "uliza", means "ask" in the local dialect of the location where the initial chatbot was launched.

2.1 Version Control and Location Management

The primary innovation compared to the current version of the *ULIZA-WI* chatbot is the ability to select versions based on user preferences and the option to input a location to receive weather data for. The three modes available are the *Data Saving Version*, the *Performance Optimised Version*, and the *Extension Officers Version*. The *Data-Saving* mode blocks the option for graphs to save the data size of the messages. This mode is tailored to users who do not have much mobile data. The *Performance Optimised Version* allows users to receive graphs and is intended for farmers. Both the *Data Saving Version* and the *Performance Optimised Version* entail simplified information, therefore more aimed at farmers than extension officers. The text will be understandable and the graphs will contain considerably more illustrations and fewer numbers than the *Extension Officers Version*'s counterpart. The *Extension Officers Version* would provide graphs with numbers and more complex data and texts with more precise information. This version is purely intended for extension officers who are experienced and able to communicate more complex information to the general public.

In the same menu, users can opt to set their location. If the city is not available in the pre-processed dataset, the system computes the geographically closest matching location. In this way, the user will receive weather information that is relevant, while not having to use a third-party app for the selection of the location [2]. The processing of the location dataset is discussed in more detail in Subsection (3.1): *Data Gathering and Processing*.

2.2 Weather Information

The application provides the current weather data (temperature and precipitation) for the selected location of the nearest recorded location. Depending on the version, the user receives forecast graphs. These graphs will differ between the *Data Saving Version* and the *Extension Officers Version*.

2.3 Weather Forecasts

Users may view temperature forecast graphs for specific periods (in weeks). Graphs are retrieved from a database, which prevents the user from waiting for a graph to generate. An example of a graph for the farmer (*Performance Optimised Version*) is shown in Figure 1.

Farmer's Weather Forecast

Day	Weather	Temp	Wind	Rain	Advice
Mon	☀	30°C	windy	not rainy	Good day for harvest
Tue	☁	28°C	not windy	rainy	Check soil moisture
Wed	□	25°C	not windy	not rainy	Rain coming – store tools
Thu	☀	32°C	not windy	rainy	Hot day – water crops early
Fri	□	26°C	windy	rainy	Rain – avoid pesticide use

Figure 1: Example of a graph that can be generated with Python.

[2] This feature can be easily restructured in such a way that the app collects the coordinates of the user, making exact weather forecasts possible. In our version, this is not structured in this way, as we aim to reduce the complexity of the program with preloaded data. Adding this feature would result in the need to load the data at the selection of the option rather than using preloaded data.

2.4 Crop Advice (Cultivation or Pest and Disease)

We first provide the general advice in text form, and some advice applies to many or all crops. In this way, the farmer may not need to click on a button for specific information, reducing the data traffic. We provide two categories of advice: *cultivation of the crop* and *prevention of pests and diseases*. The cultivation advice covers the most made mistakes and attention points per stage the crop is in to facilitate and optimise the cultivation process. The pests and diseases advice discusses ways to identify certain pests and diseases, paired with ways to prevent these. These categories of information are provided for each crop.

2.5 Market Price Information

In this header, we show the most recent market prices in USD per Kg[3]. The prices are retail prices and, if possible, oriented to the Ho Chi Minh region. This provides the farmer with a general idea of their crop's worth.

2.6 Information for Alternative Farming Techniques

Under the GAP section, users can explore alternative and sustainable farming techniques. The information about *Conservation Agriculture* is a variant of the information that is provided in the chatbot *ULIZA-WI*. The user first reads the general information for good agricultural practices to introduce the subject and its advantages. Thereafter, the user can select to read more about a step-by-step guide or the three main principles with their respective information. The information is the same across all three versions: *Data Saving*, *Performance Optimised*, and *Extension Officers*, and it does not show images. The images in the chatbot *ULIZA-WI* contained unnecessarily difficult words; the illustrations were interesting but not vital to communicate the information. The information about *Aquatic Rice Farming* discusses the findings of our research and advises the user on the implementation of it.

2.7 Notifications

We provide a synthetic menu for the users to toggle notifications for weather alerts, cultivation updates per crop, and market price updates per crop. The weather alerts notify the user when drastic adverse weather is expected. For adverse weather condition, consider floods, droughts, heat waves, and strong winds. The user can then prepare themselves and their crops accordingly. The cultivation updates send the user reminders to undertake actions when the crop is expected to transition into a new state. Such as weeding and harvesting. For the prices, notifications are provided when new prices become available. The prices are not updated daily, therefore users will not experience ”spam” notifications.

At present, this functionality operates solely in a local and synthetic environment. Therefore, it does not yet occur within the deployed application.

2.8 User Interface and Data Integration

The web app mimics the interaction a user would have with a decision-tree based chatbot. Therefore, the user interface is built as a selection menu with buttons for each branch in the decision tree. Furthermore, the app retrieves all data from a GitHub repository structured with subdirectories for CSV files, graphs, and textual data. The design enables scalability and allows for external content updates without requiring code modifications. Additionally, the design enables quick responses; all data exists already.

[3]We can easily show the VND per Kg price, since we also fetch the FX rate.

3 Proposed Chatbot: Back-End

In this section, we examine the back end of the web application. The back-end system is responsible for processing data and delivering outputs, such as graphs and numerical values, to the front end for user interaction. We discuss the data gathering and manipulation, the file *wi_utils.py* and its functions, and the Python Notebook file (*backside.ipynb*) that uploads the data to GitHub. Finally, we discuss the set of files in the subsection *Set up steps*

3.1 Data Gathering and Processing

In this subsection, we discuss the sources we used for the information. There are four different categories of the information we use, including the the advices, location data, price data, and weather data.

3.1.1 Advices

The content of the advice and information points is generally fetched from the original *ULIZA-WI* chatbot, found with large language models, or gained through the research performed by the project group. We opted for advice on crop cultivation and pest and disease control for crops, information about good agricultural practices, information about aquatic rice farming, and advice that is weather-dependent for crops. The crops we currently cover are: maize, rice, and sugarcane. This list is, naturally, extendable.

3.1.2 Location Data Collection and Selection

To eliminate the need to generate data on the call of the user, we made a selection of places that would be relevant and cover the majority of Viet Nam, to maximise the probability that a location chosen by the user is close to one of the locations we selected. This way, we only need weather data for a selection of cities, rather than all of them.

We use the GeoNames dataset (GeoNames, 2024) to access detailed information about locations in Vietnam, including population, geographic coordinates, alternate names, and classification codes. The dataset was first filtered to retain only places with a population between 100 and 10,000, as well as areas designated as farmland. This allowed us to exclude both sparsely populated and highly urbanised regions, which are less relevant for our focus on smallholder agriculture. With the greedy clustering method, we only keep one place of the places that are within a 2 km radius of each other. The location that is kept is the place with the highest population to increase the probability of relevance.

If a user is looking for a place outside this set, which is likely, we take the nearest place in terms of distance (as the crow flies) that is in the set we selected set and show the locations information. The selected places are given in Figure 2. It is visible that these spread throughout the entirety of Vietnam. When considering places like Sapa, Mu Cang Chai, and Ha Giang. we find that the coverage has a max distance of 18 km. In table 1, we show the relation between the radii we considered and the amount of cities that are in the preloaded set. The total size of the dataset had 406 places, meaning that with 2 km as filter condition there were still rudimentary places. Storing the files results in a .csv file with the size of 83 kilobytes for 404 places and it takes 4 minutes and 30 seconds to upload this data in a for loop structure. The for-loop structure is kept for simplicity, but vectorising may yield even faster times.

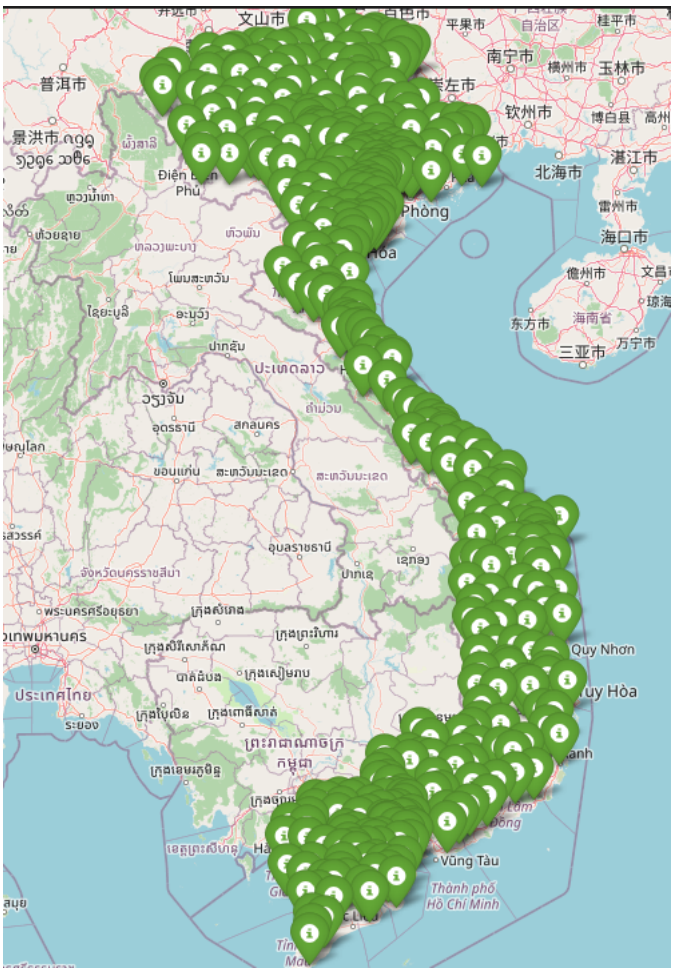

Figure 2: Map of Viet Nam, the places selected with processing are marked.

Radius (km)	Amount of Cities
30	175
25	216
20	267
15	306
10	363
5	397
2	404
1	406

Table 1: The relation between the radius needed between cities and the amount of cities that need to be considered for the preloads.

3.1.3 Crop Prices

For the price information of the crops, we used different websites. For maize, we used Selina Wamucii (2024). For sugarcane, we used Trading Economics (2024). Lastly, for rice, we used YCharts (2024)[4]. We denote the price in USD per Kg; however we can convert this to VND per KG, as we also fetch the current exchange rate with ExchangeRate-API (2024).

[4]This price assumes the rice to be five percent broken. The website was the only website that provided the price in a structured manner for web-scraping.

3.1.4 Weather Data

The current weather data is more for demonstration purposes, as we assume that Weather Impact has higher-quality current weather data and predictions. However, we still provide the sources we use for the demonstration. We use the API of *api.openweathermap.org/data* to get up-to-date weather data (OpenWeatherMap, 2025). To use this API, we need the latitude and longitude data for a location. These data points are fetched from the large dataset discussed in the paragraph *Location Data Collection and Selection* provided by GeoNames (2024). The API provides the latest temperature and precipitation levels (hourly). We use an API of *National Aeronautics and Space Administration (NASA)* for the historical weather data, this is only temperature data (NASA Langley Research Center, Prediction Of Worldwide Energy Resources (POWER) Project, 2025).

3.2 wi utils.py

The file *wi_utils.py* contains all functions we use for data processing and uploading. It is divided into four categories: Weather Data Fetching and Processing, Location Tracking, uploading text to GitHub, and fetching crop prices online. This section describes the functions, the file *wi_utils.py* contains the functions with a similar explanation per function.

3.2.1 Weather Data Fetching and Processing

This section outlines the back-end Python functions used to retrieve, process, and upload weather data for use in the web application.

get_nasa_power_weather: Retrieves historical daily temperature and precipitation data from the NASA POWER API for a specified geographic coordinate and time range.

- **Arguments:**
  - `lat` (float): Latitude of the location.
  - `lon` (float): Longitude of the location.
  - `months` (int): Number of weeks to retrieve (default is 6).
- **Returns:** A `pandas.DataFrame` with daily temperature in Celsius and precipitation in millimeters.
- **Remarks:** Raises an exception if the API request or parsing fails.

get_lon_lat_data: Resolves a city name to its geographical coordinates using a reference DataFrame.

- **Arguments:**
  - `place_name` (str): Name of the location.
  - `df` (DataFrame): Optional geographic dataset. Defaults to `coord_data`.
- **Returns:** A tuple `(latitude, longitude, population, geonameid)` or `None` if not found.
- **Remarks:** If multiple matches are found, the one with the highest population is returned.

get_weather: Fetches current weather data for a specified city using the OpenWeatherMap API.

- **Arguments:**
  - `city` (str): Name of the city.
  - `api_key` (str): Optional API key for OpenWeatherMap.
- **Returns:** A tuple `(temperature, precipitation)` in Celsius and millimeters, respectively.
- **Remarks:** Falls back to default coordinates if resolution fails. Returns `(None, None)` on error.

get_past_days_average_by_coords: Retrieves recent and average weather data (temperature, precipitation, wind speed) over a specified number of past days using OpenWeatherMap's historical hourly API.

- **Arguments:**
  - `city` (str): Name of the city to retrieve data for.
  - `days` (int, optional): Number of past days to include in the average (default is 3).
- **Returns:** A 6-element tuple `(latest_temperature, latest_precipitation, latest_wind, avg_temperature, avg_precipitation, avg_wind)` where each element is a rounded float.
- **Remarks:**
  - Falls back to default coordinates (Ho Chi Minh City) if location resolution fails.
  - Uses hourly weather data from OpenWeatherMap's History API.
  - Precipitation includes both rain and snow (if available).
  - Units are in metric: ℃ for temperature, mm for precipitation, m/s for wind speed.

gen_graphs_cities_periods: Coordinates the generation of graphs and upload of climate data for a list of cities and time periods.

- **Arguments:**
  - `selected_cities` (list): Cities to process.
  - `selected_periods` (list): Periods in weeks or months.
- **Returns:** A nested dictionary with structure {`city: {period: url`}}.
- **Remarks:** Calls `fetch_and_upload` for each city-period combination.

classify_value: Classifies a numeric value (e.g., temperature, precipitation) as below average, about average, or above average based on a comparison with a historical average and a specified tolerance.

- **Arguments:**
  - `today` (float): The current (observed) value.
  - `historical_avg` (float): The historical average to compare against.
  - `tolerance` (float, optional): Allowed deviation as a fraction of the average (default: 0.3 or 30%).
- **Returns:** A string classification:
  - `"about average"` – if within tolerance range.
  - `"above average"` – if above range.
  - `"below average"` – if below range.
  - `"undefined"` – if the historical average is zero.
- **Remarks:**
  - Handles edge case where the historical average is zero to avoid division errors.
  - Useful for contextualising daily climate data against historical norms.

collect_and_upload_weather_data: Collects weather data for a list of cities, classifies the current weather against historical averages, saves the data as a CSV, uploads it to GitHub, then deletes the local file.

- **Arguments:**

  - `cities` (list of str): List of city names to process.
  - `folder` (str, optional): Local folder path to save the CSV file temporarily (default: `"climate_data"`).
- **Returns:** `uploaded_url` (str): The URL of the uploaded CSV file on GitHub.
- **Remarks:**
  - Uses `get_past_days_average_by_coords` to retrieve weather data.
  - Uses `classify_value` to classify temperature, precipitation, and wind speed.
  - Saves the combined data with current values and classifications into a CSV file.
  - Uploads the CSV file via `upload_to_github` (assumed defined elsewhere).
  - Removes the local CSV file after uploading.
  - Requires `pandas` and `os` modules.

3.2.2 Location Tracking

In this subsection, we discuss the functions we used to retrieve locations that are predetermined by us that are closest to the locations entered by the user.

haversine_distance: Computes the great-circle distance between two geographical coordinates on Earth using the Haversine formula.

- **Arguments:**
  - `lat1`, `lon1` (float): Latitude and longitude of the first location in degrees.
  - `lat2`, `lon2` (float): Latitude and longitude of the second location in degrees.
- **Returns:** A scalar value representing the distance in kilometers between the two points.
- **Remarks:** Assumes a spherical Earth with radius 6371 km.

filter_cities_by_distance: Filters a dataset of cities by removing entries that are closer than a specified threshold distance to another, prioritising cities with larger populations.

- **Arguments:**
  - `df` (`pandas.DataFrame`): DataFrame containing city information.
  - `lat_col`, `lon_col` (str): Column names for latitude and longitude. Defaults are `"latitude"` and `"longitude"` columns.
  - `pop_col` (str): Column name for population. Default is `"population"` column.
  - `min_distance_km` (float): Minimum distance (in kilometers) required between selected cities.
- **Returns:** A filtered `DataFrame` containing cities that meet the minimum separation criterion.
- **Remarks:** Cities with higher populations are given priority in selection.

search_city: Attempts to find a city in a filtered dataset. If not found, identifies the nearest city with available data using geographical proximity.

- **Arguments:**
  - `name` (str): Name of the city to search.
  - `filtered_cities` (`DataFrame`): DataFrame containing a curated set of cities with coordinates.
  - `coord_data` (`DataFrame`): Optional full dataset of cities for backup matching (default is `viet_coord_data`).

  - `coord_threshold` (float): Maximum allowed difference in degrees for nearby city filtering. Default is 0.2.

- **Returns:** A tuple `(city_name, message)` indicating either an exact or approximate match with input place.
- **Remarks:** If an exact match is not found, the function returns the closest city based on Euclidean distance in degrees (converted to kilometers).

3.2.3 Uploading Data to GitHub

In this section, we discuss the functions that we used to upload data to GitHub. This is important as this ensures that all data that is needed by the front end already exists and can be imported easily.

upload_to_github: Uploads a local file to a specified GitHub repository path using the GitHub REST API.

- **Arguments:**
  - `filepath` (str): Local path to the file to be uploaded.
  - `repo_path` (str): Path within the GitHub repository where the file will be stored.
- **Returns:** A URL string pointing to the raw GitHub content if successful, otherwise `None`.
- **Remarks:** Requires valid global variables `GITHUB_REPO`, `GITHUB_BRANCH`, and `GITHUB_TOKEN`. The function performs either a new upload or an update depending on file existence.

fetch_and_upload: Retrieves historical weather data for a city, generates a temperature graph, and uploads it as a PNG to GitHub.

- **Arguments:**
  - `city` (str): Name of the city.
  - `months` (int): Number of months of historical data to retrieve.
- **Returns:** The raw URL of the uploaded image if successful; `None` otherwise.
- **Remarks:** Internally uses `get_lon_lat_data`, `get_nasa_power_weather`, and `upload_to_github`. The generated graph is removed from local storage after upload.

text_upload: Saves a given string of text data to a structured filename and uploads it to GitHub.

- **Arguments:**
  - `crop` (str): Crop name.
  - `type` (str): Type of content (e.g., `"description"`, `"instructions"`).
  - `text` (str): Content to upload.
- **Returns:** None.
- **Remarks:** Files are saved as `advice_{type}_{crop}.txt` and uploaded to the `texts/` directory in the GitHub repository. Temporary local files are deleted post-upload.

save_and_upload_lists: Saves a collection of Python lists to text files and uploads them to GitHub.

- **Arguments:**
  - `data_dict` (dict): Dictionary where keys are filenames (without extension), and values are the list contents.
  - `folder` (str): Folder path for local saving and remote GitHub directory (default is `"scalability"`).

- **Returns:** None.
- **Remarks:** Lists are stringified before saving and uploaded using `upload_to_github`. Local files are removed afterward.

upload_csv: Saves a DataFrame to CSV format and uploads it to the GitHub repository.

- **Arguments:**
  - `filtered_cities` (`pandas.DataFrame`): DataFrame to be uploaded.
  - `folder` (str): Local and GitHub folder for saving the file (default is `"csv_files"`).
  - `filename` (str): Name of the CSV file (default is `"filtered_cities.csv"`).
- **Returns:** The raw GitHub URL of the uploaded file.
- **Remarks:** After uploading, the local CSV file is deleted. Useful for caching geographic subsets.

save_and_upload_crop_prices: Creates and uploads a CSV file containing crop prices to a GitHub repository.

- **Arguments:**
  - `CropPrices` (dict): Dictionary mapping crop names to prices.
  - `selected_crops` (list): List of crop names to include in the CSV.
  - `folder` (str): Folder to temporarily store and upload the CSV file (default: `"price_data"`).
- **Returns:** None.
- **Remarks:** Saves crop-price data locally, uploads it to GitHub using `upload_to_github`, and removes local artifacts after completion.

plot_forecast: Generates a visual weather forecast table with icons, temperature, wind, precipitation, and farming advice, then saves and uploads the plot image.

- **Arguments:**
  - `intervals` (list of str): Time intervals for the forecast (e.g., `["Today", "Tomorrow", "3rd Day"]`).
  - `weather` (list of str): Weather conditions for each interval (e.g., `["Sunny", "Cloudy", "Rainy"]`).
  - `temp` (list of str): Temperatures corresponding to each interval (e.g., `["30°C", "28°C", "25°C"]`).
  - `wind` (list of str): Wind speeds or descriptions (e.g., `["5 km/h", "10 km/h"]`).
  - `precip` (list of str): Precipitation levels (e.g., `["0 mm", "2 mm", "5 mm"]`).
  - `advice` (list of str): Farming advice aligned with each forecast (e.g., `["Harvest today", "Avoid pesticide use"]`).
  - `name` (str, optional): Base name for the saved image file (default: `"weather_forecast_example"`).
- **Returns:** None. Saves a PNG image and uploads it to the `graphs/` directory in the GitHub repository.
- **Remarks:**
  - Automatically includes emoji-style weather icons (sunny icon, cloudy icon, rain icon).
  - Saves the figure using `matplotlib` and uploads via `upload_to_github()`.
  - Assumes that the GitHub upload function and target directory structure are correctly configured.

3.2.4 Fetching Crop Prices Through Web-Scraping

In this part, we discuss the functions we used to obtain relevant prices for the crops.

fetch_vietnam_rice_price: Retrieves the most recent price of Vietnam 5% Broken Rice from `ycharts.com`, expressed in USD per kilogram.

- **Arguments:** None.
- **Returns:** A `float` indicating the rice price in USD/kg or a `str` error message if retrieval fails.
- **Remarks:** Parses the web content using XPath and converts the unit from USD/ton to USD/kg.

fetch_sugar_price: Extracts the latest sugar price in USD per kilogram by scraping `tradingeconomics.com`.

- **Arguments:** None.
- **Returns:** A `float` for the converted price or a `str` error message upon failure.
- **Remarks:** Parses JSON-like metadata embedded in the site's JavaScript and performs unit conversion from cents per pound.

get_vnd_to_usd_rate: Retrieves the exchange rate of Vietnamese Dong (VND) to US Dollar (USD) via the ExchangeRate API.

- **Arguments:**
  - `API_KEY` (str): Optional. API key for `exchangerate-api.com`.
- **Returns:** A `float` representing the VND to USD exchange rate, or `None` if retrieval fails.
- **Remarks:** Performs error checks on the API response and ensures valid currency conversion data.

fetch_vietnam_maize_price: Retrieves and averages retail maize prices from `selinawamucii.com`, then converts from VND/kg to USD/kg.

- **Arguments:** None.
- **Returns:** A `float` for the average maize price in USD/kg, or `None` on failure.
- **Remarks:** Utilises XPath scraping, regex pattern matching for numeric values, and calls `get_vnd_to_usd_rate` for conversion.

3.3 backside.ipynb

This section provides an overview of the code functionality implemented in the `backside.ipynb` notebook. The code supports a web-based application designed to fetch, process, visualise, and upload agricultural and meteorological data, mainly for Vietnam, using open APIs and GitHub.

3.3.1 Importing the Libraries and setting the crop selection

The notebook begins by importing the functions that we defined in the file *wi_utils.py*. In the notebook, we primarily use the pre-defined functions. We also set the crop selection, the crops we cover are *selected_crops = ["Rice", "Maize", "Sugarcane"]*.

3.3.2 Data Retrieval

The raw geographic data for Vietnamese settlements is first loaded from the GeoNames file `VN.txt` using predefined column headers. After uploading the unprocessed dataset to GitHub as a complete dataset, we keep only relevant information such as latitude, longitude, population, and elevation. These are converted to numeric types, and rows with missing or invalid values are discarded. The filtering procedure targets small to mid-sized rural settlements by selecting entries from feature class `P` (e.g., `PPL`, `PPLA2`) with populations between 100 and 10,000, along with select agricultural sites classified as `FRM`. To improve geographic diversity, a greedy clustering heuristic excludes locations within 30 km of already-selected cities, prioritising higher population. The result is a selection of locations that covers a large part of Viet Nam. Finally, Ho Chi Minh City is added explicitly. Lastly, the cleaned dataset is uploaded as `filtered_cities.csv`.

To visualise the filtered set of the selected locations, the function `plot_places_on_vietnam_map` generates an interactive map using the `folium` library. It computes the geographical bounds of the selected cities and centres the map accordingly. Each city is plotted as a marker labelled with its name, using a green icon for visibility. The map view is then automatically adjusted to fit all markers within the display window.

3.3.3 Uploading Data to GitHub

In the last part, relevant data is uploaded to GitHub. We start with the now processed list of relevant location, the selected crops, and the periods for the forecasts. These will be sent in the form of a .csv file.

Uploading the Dynamic Data
We use the fetching functions for the temperature and prices to retrieve and upload the dynamic data to GitHub. These functions should be run periodically.

Uploading Static Data
In the last part of the file, we use a text write function to upload the static data, such as ”Alternate Farming Techniques”, and ”Crop Advice” to GitHub. We enter the data we want to show manually, and use the respective filename to direct the file to the correct folder. This part, naturally, only needs to be run at updates.

3.4 Set Up Steps

We use Python libraries for the files. In this subsection, we discuss the different Python files and the set up of the *.env* file. *wi_utils.py*

```
#time manipulation
from datetime import datetime, timedelta
from dateutil.relativedelta import relativedelta
import time

#the data manipulation
import pandas as pd
import numpy as np

# Plotting tools
from matplotlib.patches import Rectangle
import matplotlib.pyplot as plt

# interaction with external APIs and files
import re
import os
import json
from dotenv import load_dotenv
from lxml import html
import requests
```

For the file *backside.ipynb*, the interaction file.

```
#interact with the internet
import os

#data manipulation
import pandas as pd

# load the external library that is created,  this is the wi_utils.py file
import importlib
import wi_utils
from wi_utils import *
importlib.reload(wi_utils)
```

For the front-end file, *test_2.py*, we use:

```
# the front end interaction
import streamlit as st
from functools import partial

#interaction with the internet
import requests
import ast

#the data manipulation
import pandas as pd
import numpy as np
```

Since we use API's, we load API keys locally with an *.env* file and the *load_dotenv* function from the *dotenv* library. The *.env* file is structured as

```
GITHUB_REPO = "" #username/repository_name
GITHUB_BRANCH = "" #branc name, usually "main"
GITHUB_TOKEN = "" #github access token for the respective repository
weather_API_KEY = "" # the API key for the api.openweathermap.org/data API
fx_API_KEY = "" #the API key for v6.exchangerate-api.com/v6
```

4 Concluding Remarks

The backend architecture of the *HOI-WI* application entails a modular and extendable system for a DCAS Chatbot, tailored towards smallholder farmers in Viet Nam. The novelty of the approach lies in the combination of real-time and historical data aggregation from multiple public APIs, automated preprocessing of geographic and weather data, and integration with GitHub for version-specific storage. Through the use of location clustering and distance heuristics, the application achieves spatial coverage while maintaining computational efficiency. Additionally, the structured methodology for price acquisition via web scraping ensures relevant price data for users. These components, collectively, represent a significant step toward accessible, data-driven agricultural advisories with minimal infrastructure demands and high adaptability to local contexts.

References

ExchangeRate-API (2024). Exchangerate-api. https://v6.exchangerate-api.com/v6/{API_KEY}/latest/USD. https://www.exchangerate-api.com/.

GeoNames (2024). Geonames - vietnam data for locations. Accessed: 2025-07-02.

NASA Langley Research Center, Prediction Of Worldwide Energy Resources (POWER) Project (2025). NASA power data access viewer (dav). https://power.larc.nasa.gov/data-access-viewer/. Accessed: 2025-07-04.

OpenWeatherMap (2025). OpenWeatherMap api. https://openweathermap.org/api. Accessed: 2025-07-04.

Selina Wamucii (2024). Selina wamucii - global food market insights. https://www.selinawamucii.com/. Accessed: 2025-07-04.

Trading Economics (2024). Sugar commodity price. https://tradingeconomics.com/commodity/sugar. Accessed: 2025-07-04.

YCharts (2024). Vietnam 5% broken rice price. https://ycharts.com/indicators/vietnam_5_broken_rice_price. Accessed: 2025-07-04.